

In react app, when the first time a component is rendered, it means it is mounted onto DOM?

Yes — in a React app, when a component is rendered for the first time, that *does* correspond to it being mounted onto the DOM.

Here's the breakdown:

✅ Mounting

- **Mounting** is the phase when a component instance is created *and* inserted into the DOM.
- This happens the first time React calls the component and places its output into the DOM tree.

In function components:

- The component function runs for the first time.
- React applies the returned JSX to the DOM.
- The `useEffect(() => {}, [])` hook (with empty deps) runs *after* the component has been mounted.

In class components:

- `constructor()` runs.
- `render()` runs.
- `componentDidMount()` runs after the component is mounted.

🧐 Subtle detail

"Rendered" can sometimes mean either:

1. React calling the component function (i.e., virtual render), or
2. The DOM actually updating (i.e., commit phase).

But in normal conversation, "first render" means when the component is mounted.

✅ Therefore:

Yes, first render = component mount.

If you'd like, I can also explain the reconciler/commit phases or show an example.



If I pass nothing inside the dependency array in `useEffect`, does it mean it will be called only once per mounting?

Yes — if you pass an **empty dependency array** (`[]`) to `useEffect` , it will run **only once per component mount**.

✅ What `useEffect(() => { ... }, [])` means

- The effect runs **after the component is mounted**.
- It will **never run again** unless the component is unmounted and later mounted again.
- It will **not** run on re-renders.

✅ Example

jsx

Copy code

```
useEffect(() => {  
  console.log("Runs only once – on mount");  
}, []);
```

🔧 What about cleanup?

If your effect returns a cleanup function, like:

jsx

Copy code

```
useEffect(() => {  
  console.log("Mounted");  
  
  return () => {  
    console.log("Cleanup on unmount");  
  };  
}, []);
```

- The cleanup runs **once on unmount**.
- The effect itself runs **once on mount**.

⚠️ Note about React Strict Mode (dev-only)

If your app is wrapped in `React.StrictMode`, React intentionally **runs the effect twice in**

development to detect side-effect issues.

But this happens **only in dev**, not in production.

So, aside from Strict Mode's dev behavior:

✅ **Empty dependency array → effect runs exactly once per mount**



If I pass a state variable inside the dependency array, does it mean `useEffect` will be called every time the component is rendered, since changing state variable causes re-rendering? Also, will the cleanup function be called on every render?

Great questions — let's break it down clearly.

✅ **1. If you put a state variable in the dependency array, will the effect run every time the component renders?**

Not necessarily.

👉 The effect runs **only when the dependency *changes*** — not on every render.

Example

jsx

Copy code

```
useEffect(() => {  
  console.log("Effect runs");  
}, [count]);
```

This effect runs when:

- the component mounts (first render)
- `count` **changes** (not just any render)

If the component re-renders for other reasons (e.g., a different state update), the effect will **NOT** run unless `count` actually changed.

✅ 2. Will the cleanup function run every time the effect runs?

Yes.

Effect lifecycle:

1. On mount: effect runs.
2. On dependency change:
 - cleanup runs
 - then the effect runs again
3. On unmount: cleanup runs one last time

Example

jsx

Copy code

```
useEffect(() => {
  console.log("Effect");

  return () => {
    console.log("Cleanup");
  };
}, [count]);
```

If count changes from 1 → 2 → 3:

You get:

scss

Copy code

```
Effect (on mount)
Cleanup (before re-running effect)
Effect (after count changed)
Cleanup
Effect
...
```

So yes:

🔥 Cleanup runs before every re-run when dependencies change
and

🔥 Cleanup runs on unmount.

Summary Table

Scenario	Effect runs?	Cleanup runs?
First mount	✅ Yes	❌ No
Dependency changes	✅ Yes	✅ Yes (before re-run)
Re-render without dependency change	❌ No	❌ No
Unmount	❌ No	✅ Yes

If you'd like, I can illustrate this with logs or a visual flowchart.

